

slime 上手实操与技术详解

面向 THUDM/slme 的工程化读本：从 Docker 跑通第一轮 GRPO，到解释 Megatron 训练、SGLang rollout、Ray 编排、数据缓冲、优势估计、损失计算、权重同步、低精度、容错和扩展接口。

Winston

资料口径截至 2026-05-06

THUDM/slme main@82007faf · latest release v0.2.4

[Official repository](#) · [documentation](#) · [source code](#) · [release notes](#)

| 目录

01	阅读前的定位	slime 解决什么问题
02	一小时跑通	Docker、模型、数据、转换、启动脚本
03	脚本拆解	资源、批量、Megatron 与 SGLang 参数
04	系统架构	training、rollout、data buffer、Ray
05	源码级训练闭环	train.py 与 train_async.py
06	Rollout 生成链路	SGLang router、Sample、reward、partial rollout
07	训练端计算	logprob、advantage、policy loss、value loss
08	权重同步与显存切换	colocate、decoupled、NCCL、Ray IPC
09	扩展与生产要点	自定义函数、低精度、容错、调试、边界

阅读前的定位

slime 是 THUDM 开源的 LLM post-training 框架，官方描述为面向 RL Scaling 的后训练系统。它的核心价值不在于把 PPO 或 GRPO 写成一个训练脚本，而在于把大规模强化学习后训练所需的训练引擎、推理服务、样本生成、奖励计算、数据缓冲、权重同步、资源编排和可扩展接口放到同一个工程闭环中。

SYSTEM ROLE

训练与推理的双引擎系统

训练侧以 Megatron 为主，负责 actor、critic、reference、teacher、optimizer 和并行训练；rollout 侧以 SGLang 与 sgl-router 为主，负责高吞吐生成、路由、KV 管理、奖励与样本组织。

PRIMARY OBJECTIVE

让 RL 扩展到真实大模型

slime 针对的不是单卡算法 demo，而是 GLM、Qwen、DeepSeek、Llama 等大模型后训练中常见的多 GPU、多节点、长响应、动态采样、低精度 rollout 和权重同步问题。

DESIGN TRAIT

SGLang-native 与 Megatron-native

slime 对 SGLang 参数使用 `--sglang-` 前缀透传，对 Megatron 参数保持原生风格。训练循环没有被深度封装进不可见的 trainer，而是保留在 `train.py` 与 `train_async.py`。

SNAPSHOT

main@82007faf

本文基于 2026-05-06 抓取的上游快照撰写。对应最新 release 为 v0.2.4，发布于 2026-03-29；仓库当日 HEAD 为 82007faf4b398abd32bd8e07f9638f6cfeb70729。

slime 不应被误读成什么

第一，它不是一个只提供 reward model glue code 的轻量脚手架。源码中真正复杂的部分，是 rollout server 的生命周期、Ray placement group 的资源隔离、Megatron actor 的训练数据整理、策略损失的分布式计算，以及训练权重向 SGLang 推理引擎的同步。

第二，它不是一个通用推理服务框架。SGLang 是 slime 的 rollout 引擎，但 slime 的上层职责是后训练闭环。若目标只是部署聊天 API，直接使用 SGLang、vLLM 或 TensorRT-LLM 更合适；若目标是把生成、判分、训练、保存、评估和权重更新串成可扩展的 RL pipeline，slime 才是合适的入口。

第三，它不是隐藏底层复杂度的低代码平台。slime 保留了大量 Megatron、SGLang、Ray、NCCL 和模型配置参数。使用者需要理解 tensor parallel、pipeline parallel、expert parallel、context parallel、rollout batch、global batch、显存分配、checkpoint 格式和 tokenizer 约束，否则脚本可以启动，但训练语义未必正确。

一条主线理解 slime

最稳妥的理解方式，是把 slime 看作一个分布式控制器。它先从 prompt 数据源取出一批问题，在 SGLang 集群上生成多个候选响应；然后把响应、token、loss mask、reward、rollout logprob、metadata 等结构化为 Sample；再按数据并行切分为 Megatron 可消费的训练批；训练端计算 ref logprob、teacher logprob、优势、policy loss 或 value loss；optimizer 更新 actor 后，slime 把新权重同步回 SGLang rollout 引擎，下一轮 rollout 使用更新后的策略继续采样。

READING METHOD

本文按工程落地顺序组织：先跑通，再拆参数，再读架构，最后进入源码链路。这样做的原因很简单：slime 的学习曲线不是 API 名称，而是训练侧与生成侧的资源耦合。只看算法公式，无法解释为什么 $\text{rollout-batch-size} \times \text{n-samples-per-prompt}$ 必须匹配 $\text{global-batch-size} \times \text{num-steps-per-rollout}$ ；只看启动脚本，也无法判断权重同步和样本过滤是否符合任务目标。

一小时跑通

slime 的官方快速开始以 Docker 为主路径。对首次上手而言，不建议先在裸机拼装环境；Megatron、SGLang、CUDA、NCCL、Ray、TransformerEngine、模型转换工具和 tokenizer 版本任何一处偏差，都可能把问题从“理解 slime”变成“排查深度学习环境”。

环境准备

官方镜像为 `slimerl/slime:latest`。容器启动时应打开 GPU、IPC 和较大的共享内存；训练与推理进程之间会使用 NCCL、Ray 和多进程通信，默认 Docker 配置往往不足。

```
docker pull slimerl/slime:latest

docker run --rm --gpus all --ipc=host --shm-size=16g \
  --ulimit memlock=-1 --ulimit stack=67108864 \
  -it slimerl/slime:latest /bin/bash
```

容器内通常已经包含 slime。若需要对齐上游最新代码，可以在容器内更新源码并以 `editable` 模式重新安装。

```
cd /root/slime
git pull
pip install -e . --no-deps
```

HARDWARE NOTE

slime 的示例脚本假定存在可用于 Megatron 训练和 SGLang rollout 的多张 GPU。单卡或显存较小的消费级 GPU 可能无法直接运行官方 GLM/Qwen 示例。此处的目标是说明标准路径，生产训练前仍需按模型规模、上下文长度、并行策略和 rollout 并发重新预算显存。

下载模型、训练数据和评估数据

官方快速开始以 `zai-org/GLM-Z1-9B-0414`、`zhuzilin/dapo-math-17k` 和 `zhuzilin/aime-2024` 为示例。模型下载到本地后，需要同时满足两条链路：Megatron 训练侧需要 Megatron checkpoint，SGLang rollout 侧需要 Hugging Face 格式模型目录和 tokenizer。

```
cd /root
git lfs install
git clone https://huggingface.co/zai-org/GLM-Z1-9B-0414
git clone https://huggingface.co/datasets/zhuzilin/dapo-math-17k
git clone https://huggingface.co/datasets/zhuzilin/aime-2024
```

把 Hugging Face checkpoint 转成 Megatron torch_dist

Megatron 训练侧不能直接读取 Hugging Face checkpoint。slime 推荐的训练 checkpoint 格式是 `torch_dist`。转换前必须 source 对应模型的参数脚本，使 hidden size、层数、attention heads、group query、vocab size、rotary 参数和并行配置与模型真实结构一致。

```
cd /root/slime
source scripts/models/glm4-9B.sh

PYTHONPATH=/root/Megatron-LM python tools/convert_hf_to_torch_dist.py \
  ${MODEL_ARGS[@]} \
  --hf-checkpoint /root/GLM-Z1-9B-0414 \
  --save /root/GLM-Z1-9B-0414_torch_dist
```

训练结束后若要导出 Hugging Face 格式，可从某个 Megatron 迭代目录反向转换。这里的 `--origin-hf-dir` 不是装饰参数，它提供 tokenizer、config 与 HF 目录结构基准。

```
PYTHONPATH=/root/Megatron-LM python tools/convert_torch_dist_to_hf.py \
  --input-dir /path/to/torch_dist_ckpt/iter_xxx/ \
  --output-dir /root/GLM-Z1-9B-0414-iter_xxx \
  --origin-hf-dir /root/GLM-Z1-9B-0414
```

启动官方示例

GLM-4 9B 示例可直接通过脚本启动。这个命令背后会创建 Ray placement group，启动 SGLang rollout server 与 router，初始化 Megatron actor，执行首轮权重同步，然后进入 generate、train、save、update weights 的循环。

```
cd /root/slime
bash scripts/run-glm4-9B.sh
```

若使用 Qwen3-4B，官方脚本走 colocate 路径，更适合在同一批 GPU 上交替运行训练与 rollout。脚本会自动检测 GPU 数量，并设置 `--colocate` 与 `--sglang-mem-fraction-static 0.7`。

```
cd /root/slime
bash scripts/run-qwen3-4B.sh
```

成功启动后应观察什么

观察点	含义	异常时优先排查
Ray actor 创建	训练 actor、rollout engine、router 与辅助 actor 被调度到预期 GPU。	GPU 数量、Ray 资源、placement group、端口占用。
SGLang server ready	rollout HTTP server 已加载模型、tokenizer 与 processor，并注册到 router。	<code>--hf-checkpoint</code> 、显存、SGLang 参数、模型架构支持。
首次 weight update	Megatron actor 权重同步到 rollout 引擎，rollout 才使用正确策略。	checkpoint 转换、TP/PP/EP/CP 配置、NCCL、colocate/decoupled 模式。
rollout samples	prompt 被扩展为多个 Sample，包含 response、reward、loss mask 和状态。	数据格式、chat template、reward function、生成长度、过滤器。
training metrics	policy loss、reward、response length、KL、clip ratio 等开始上报。	batch 等式、advantage estimator、ref logprob、loss mask。

MINIMAL CORRECTNESS CHECK

第一次跑通不要只看进程是否存活。更重要的是确认：rollout 使用的是刚同步的 actor 权重；reward 与 loss mask 不为空；训练端看到的 token 与 response length 合法；首个训练 step 的 KL 与 reference 关系符合预期。slime 的发布说明中也将“首个 rollout 的 recomputed log prob 与 reference log prob 一致、首个训练 step 的 ppo_kl 为 0”作为正确性 CI 的关键检查思路。

脚本拆解

slime 的启动脚本同时配置四类系统：资源编排、Megatron 训练、SGLang rollout、RL 算法与数据。阅读脚本时不宜按参数出现顺序理解，而应按“这组参数控制哪条链路”拆解。

最重要的批量等式

官方文档明确要求：

$$\text{rollout-batch-size} \times \text{n-samples-per-prompt} = \text{global-batch-size} \times \text{num-steps-per-rollout}$$

左侧表示每轮 rollout 生成多少条可训练样本，右侧表示 Megatron 在这一轮 rollout 数据上消耗多少样本。若左侧大于右侧，样本会滞留或被截断；若左侧小于右侧，训练端会等待或重复消费不符合预期的数据。以官方 GLM/Qwen 示例常见配置为例，rollout-batch-size=32，n-samples-per-prompt=8，左侧为 256；若 global-batch-size=256 且 num-steps-per-rollout=1，训练与生成正好对齐。

资源参数

参数	控制对象	工程含义
--actor-num-nodes	Megatron actor	训练侧节点数量。影响数据并行、tensor parallel、pipeline parallel 和 NCCL 拓扑。
--actor-num-gpus-per-node	Megatron actor	每节点训练 GPU 数。与 Megatron 并行参数共同决定 world size。
--rollout-num-gpus	SGLang rollout	rollout 总 GPU 数。decoupled 模式下通常独立于训练 GPU，colocate 模式下与 actor 共享。
--rollout-num-gpus-per-engine	SGLang engine	单个 SGLang engine 使用多少 GPU，通常对应 rollout tensor parallel 或 engine 内并行。
--colocate	资源布局	训练和 rollout 共享同一组 GPU，依靠 offload、onload 和显存释放在两个阶段切换。
--prefill-num-servers	PD disaggregation	使用 prefill/decode 分离时的 prefill server 数量，用于长上下文和 agentic RL 场景。

Megatron 参数

Megatron 参数主要来自模型脚本，例如 scripts/models/glm4-9B.sh 或 scripts/models/qwen3-4B.sh。这些参数描述模型结构和训练并行策略。Qwen3-4B 脚本中可以看到 36 层、hidden size 2560、FFN hidden size 9728、32 个 attention heads、8 个 query groups、vocab size 151936、rotary base 1000000 等结构参数；这些值必须与 checkpoint 匹配。

类别	常见参数	错误后果
模型结构	<code>--num-layers</code> 、 <code>--hidden-size</code> 、 <code>--ffn-hidden-size</code> 、 <code>--num-attention-heads</code>	权重形状不匹配，或者训练虽然启动但语义错误。
并行策略	<code>--tensor-model-parallel-size</code> 、 <code>--pipeline-model-parallel-size</code> 、 <code>--context-parallel-size</code> 、 <code>--expert-model-parallel-size</code>	NCCL group、参数切分、sequence 切分或 expert 权重聚合错误。
批量与长度	<code>--global-batch-size</code> 、 <code>--micro-batch-size</code> 、 <code>--seq-length</code> 、 <code>--max-tokens-per-gpu</code>	显存溢出、padding 过多、数据并行切分不均或训练吞吐异常。
checkpoint	<code>--ref-load</code> 、 <code>--load</code> 、 <code>--save</code>	reference、actor 初始化或保存目录不正确。

SGLang 参数

SGLang 参数在 slime 中以 `--sglang-` 前缀传入。使用者应把它们视为 rollout 引擎参数，而不是训练参数。`--hf-checkpoint` 用于初始化 SGLang server 和 tokenizer；真正的 actor 权重会在训练循环开始前由 Megatron 同步到 SGLang。因此，HF checkpoint 与 Megatron checkpoint 需要来自同一模型族和同一 tokenizer 语义。

参数	建议理解	常见用途
<code>--hf-checkpoint</code>	SGLang 初始化目录	加载 tokenizer、processor、config 和初始权重外形。
<code>--sglang-context-length</code>	rollout 上下文长度	控制 prompt 与 response 的总长度上限。
<code>--sglang-mem-fraction-static</code>	静态显存比例	colocate 或显存紧张时控制 SGLang 的显存预留。
<code>--sglang-server-concurrency</code>	请求并发	影响 GenerateState 中的 semaphore，并约束并发生成。
<code>--router-*</code>	sgl-router 参数	透传到 router，控制路由行为、metrics 和 worker 注册。

RL 与数据参数

slime 支持多种 advantage estimator，包括 grpo、gspo、reinforce_plus_plus、reinforce_plus_plus_baseline 和 ppo。官方 GLM/Qwen 示例采用 GRPO，并通过 `--rm-type deepscaler` 指向数学任务 verifier。数据采用 JSONL，每行是一个 JSON object，可通过 `--input-key`、`--label-key`、`--apply-chat-template` 和 `metadata` 字段控制样本解析。

```
{
  "prompt": "计算 23 + 19，并给出推理过程。",
  "label": "42",
  "metadata": {
    "source": "demo-math"
  }
}
```

对于真实任务，reward function 的质量比启动脚本更重要。slime 提供了 `--custom-rm-path`、`--custom-generate-function-path`、`--dynamic-sampling-filter-path`、`--buffer-filter-path`、`--custom-loss-function-path` 等接口，允许把任务逻辑放入 pipeline，而不是修改框架主干。

系统架构

slime 的架构可以压缩成三个模块：training、rollout、data buffer。training 侧负责 Megatron 训练与权重产出；rollout 侧负责 SGLang 生成、奖励和样本状态；data buffer 侧负责 prompt 初始化、样本流转和局部回收。Ray 将这些角色编排为分布式 actor，并通过 placement group 将 GPU 资源固定到合理位置。

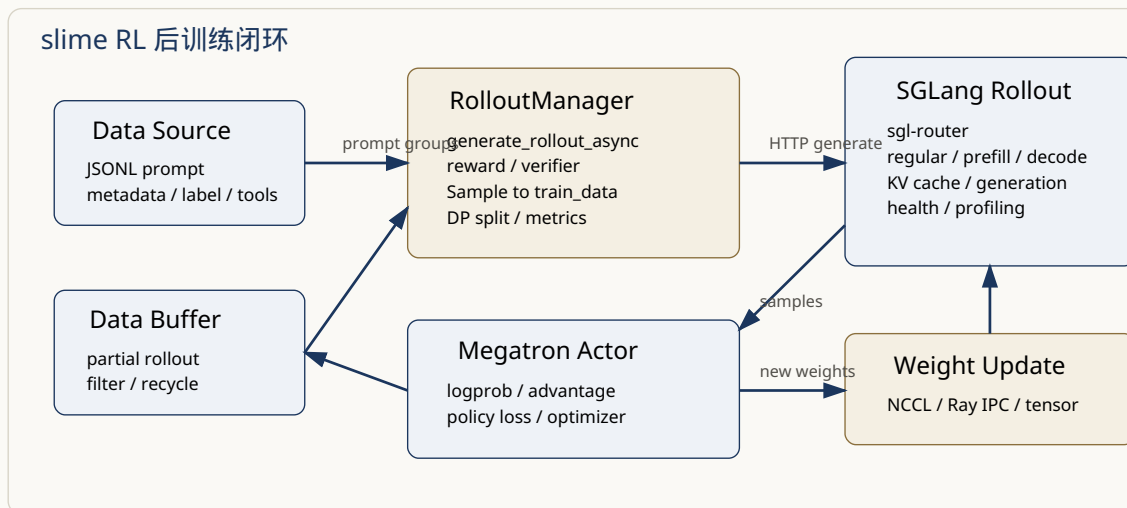


图 1: slime 的主控制闭环。图中箭头表示数据或权重的主要流向，真实实现中由 Ray actor、HTTP server、NCCL group 和本地进程共同完成。

Ray placement group 的作用

`slime/ray/placement_group.py` 会创建 Ray placement group，并通过 `InfoActor` 收集节点 IP 与 GPU ID，再按节点和 GPU 排序。这一步看似普通，实际上决定了训练 actor 与 rollout engine 的资源映射稳定性。对于多节点训练，GPU 的物理位置、节点分布和 Ray bundle 排列会影响 NCCL 通信、SGLang server 分布和 colocate 模式下的显存切换。

若不使用 `--colocate`，rollout GPU 通常从 actor GPU 之后开始分配，训练与推理资源相对独立。若使用 `--colocate`，训练与 rollout 共享 GPU，框架必须在训练和生成之间释放、恢复、offload 或 onload 显存。Qwen3-4B 示例就采用 colocate 路径，并通过 SGLang 的 `static memory fraction` 控制 rollout 侧显存占用。

训练模型的创建

`create_training_models` 会创建 actor 和可选 critic，并为不同角色应用 Megatron 参数 override。actor 是核心训练对象，critic 只在需要 value function 的算法中启用。创建完成后，rollout manager 会被附着到训练 actor，后续 actor 执行 `update_weights()` 时可以直接访问可更新的 rollout engines。

rollout server 的结构

`slime/ray/rollout.py` 中的 `ServerGroup` 与 `RolloutServer` 管理 SGLang 进程。一个 model 可以有多个 server group, group 类型包括 `regular`、`prefill`、`decode`、`placeholder` 和 `encoder`。regular 适合普通 rollout; prefill/decode 用于 PD disaggregation; placeholder 和 encoder 支持更复杂的多模型或多模态配置。

router 由 `sglang_router.launch_router.RouterArgs` 启动。slime 已经将路由栈统一到 `sgl-router`, 并在 v0.2.4 release 中明确移除了旧的 `slime-router`。router 的职责不是训练, 而是把 rollout 请求分发到正确的 SGLang workers, 提供 worker 注册、请求转发、指标和健康检查入口。

源码级训练闭环

slime 的主训练循环保留在 `train.py` 中。这个选择值得重视：大规模 RL 后训练的控制流经常需要按任务改动，例如何时评估、何时保存、何时更新权重、何时 offload rollout、何时 onload KV cache。如果框架把这些步骤封进深层 `trainer`，调试难度会显著上升。

train.py 每个 rollout_id 的主要顺序



colocate / offload 情况下，generate 与 train 之间还会穿插 rollout 权重、KV cache、训练权重和显存占用的 release / resume / 异步训练则由 `train_async.py` 提前启动下一轮 rollout，再在权重同步点等待 generation future。

图 2: 同步训练循环与资源切换。真实源码中还包含 tracking、metrics、fault tolerance、profiling 和分布式 barrier。

同步训练路径: train.py

`train.py` 的关键顺序如下。首先配置日志、初始化 tracking，创建 placement groups 和 rollout manager；随后创建 actor/critic 并执行一次 `actor_model.update_weights()`，确保 SGLang rollout engine 使用当前 actor 权重。进入循环后，每个 `rollout_id` 可先执行评估，再通过 `rollout_manager.generate` 生成训练数据；训练端消费这批数据，更新 actor 和可选 critic；保存 checkpoint 后，框架在需要时清理训练显存、恢复 rollout 侧权重和 KV cache，并再次同步 actor 权重。

这个顺序解释了一个常见误区：rollout 不是“训练前的独立数据预处理”，而是每轮策略更新的组成部分。对于 on-policy 或近似 on-policy 的 RL 算法，样本必须与当前或可追踪版本的 policy 关联。slime 的 `Sample` 中保留 `weight_versions`，rollout logprobs 也可被保存并用于训练侧校验或修正。

异步训练路径: train_async.py

`train_async.py` 的目标是隐藏 rollout 或训练的部分等待时间。它不支持 colocate，因为异步路径要求训练与 rollout 同时占用资源。核心差异是：框架可以先启动下一轮 rollout，再训练当前数据；当达到 `update_weights_interval` 或必须同步时，再等待 generation future 并更新 rollout engine 权重。

异步训练提高资源利用率，但也引入 policy staleness。是否采用异步路径，取决于任务是否能容忍样本来自略旧的策略、训练吞吐与 rollout 吞吐哪个更慢、以及 weight update 的成本是否足够高。对严格 on-policy 实验，默认同步路径更易解释；对大模型长响应训练，异步路径可能更接近可接受的工程折中。

RayTrainGroup 与 MegatronTrainRayActor

`slime/ray/actor_group.py` 中的 `RayTrainGroup` 会为每张训练 GPU 创建一个 `MegatronTrainRayActor`。`actor` 启动前会设置一组环境变量，例如 `NCCL_CUMEM_ENABLE`、`NVTE_FP8_BLOCK_SCALING_FP32_SCALES`，以及 `offload` 训练时可能需要的 `LD_PRELOAD`。这些不是普通配置噪声，而是为 `NCCL`、`TransformerEngine`、`FP8 block scaling` 和 `torch memory saver` 准备运行时环境。

`MegatronTrainRayActor.init` 会初始化分布式通信、Megatron 全局参数、HF config/tokenizer、模型、optimizer、scheduler、weight backuper 和可选 reference/teacher/old_actor。随后根据资源布局选择权重更新器：`colocate` 模式使用 `UpdateWeightFromTensor`，`decoupled` 模式使用 `UpdateWeightFromDistributed`。

训练数据进入 Megatron 前发生了什么

`_get_rollout_data` 是训练侧的关键入口。它将 rollout 产生的 token、response length、loss mask、reward、metadata、multimodal input、rollout logprob 和 routed experts 整理为 GPU 上的训练张量。若使用 `context parallel`，会对 sequence 维度进行切分；若 QKV 格式为 `bshd`，还要处理 padding；若存在 teacher logprob 或 expert routing 信息，也会按并行策略整理。

因此，rollout 输出的每一个字段都不是孤立 JSON。它会进入 Megatron 的 packed sequence、loss mask、优势估计和损失函数。自定义 rollout 或 reward 时，必须遵守 `Sample` 的字段语义，否则错误可能直到 loss 计算时才暴露。

Rollout 生成链路

rollout 是 slime 的工程重心之一。大模型 RL 的昂贵部分不只是反向传播，生成长响应、动态过滤、奖励计算、失败恢复、partial rollout、权重版本管理和多模型路由同样决定训练效率。

Sample 是核心数据结构

slime/utils/types.py 中的 Sample 将 prompt、tokens、response、response_length、label、reward、loss_mask、rollout_log_probs、teacher_log_probs、metadata、train_metadata、multimodal_inputs、session_id、status、timing 和权重版本组织在一个对象中。它的状态枚举包括 pending、completed、truncated、aborted 和 failed。

这个结构的设计目的，是让生成、奖励、过滤、训练和日志共享同一份样本语义。比如动态采样可以根据 reward 分布过滤样本，partial rollout 可以把 aborted 请求保留到 buffer，训练端可以用 loss mask 控制哪些 token 参与 policy gradient，多模态任务可以携带 processor 所需输入。

GenerateState 与并发控制

slime/rollout/sglang_rollout.py 中的 GenerateState 是 rollout 过程的单例状态。它保存 tokenizer、processor、采样参数、HTTP client、server URL 和 semaphore。并发上限由 $\text{sglang_server_concurrency} \times \text{rollout_num_gpus} / \text{rollout_num_gpus_per_engine}$ 计算，保证请求数量与 rollout engine 数量和单 engine GPU 数匹配。

生成函数会准备 prompt ids，将请求发送给 router 的 /generate 接口，并要求返回 logprob。若启用 routed experts 或多模态输入，也会将相关字段传入。响应返回后，Sample 会被更新为完整 token 序列、response 文本、response_length、rollout_log_probs、finish reason 与 meta info。

自定义 generate 与 reward

默认生成路径适合普通 prompt-response 任务。若任务需要多轮交互、工具调用、环境反馈或特殊采样逻辑，应通过 `--custom-generate-function-path` 注入异步生成函数。

```
async def generate(args, sample: Sample, sampling_params) -> Sample:
    ...
    return sample
```

reward 逻辑可通过 `--custom-rm-path` 或内置 reward type 接入。对于数学任务，reward 往往来自 verifier；对于 agentic 任务，reward 可能来自环境返回、规则检查、执行结果或外部服务。slime 的设计允许 reward 在 rollout 端完成，再把标量或更复杂 metadata 带入训练端。

generate_rollout_async 的循环

generate_rollout_async 会持续从 data source 取 prompt group，直到达到目标 rollout batch。每个 prompt 会扩展出 `n_samples_per_prompt` 个 Sample。生成完成后，框架应用动态过滤器，记录首尾样本和统计指标，并在需要时调用 abort 接口终止未完成请求。若启用 `--partial-rollout`，未完整生成但有价值的中间样本可进入 data buffer，供后续回收。

DataSource 与 buffer

RolloutDataSource 从 JSONL 或 dataset 加载 prompt, 并按 epoch shuffle。它记录 epoch 与 sample offset, 使 checkpoint 后恢复训练时可以继续取数。RolloutDataSourceWithBuffer 在此基础上加入 FIFO buffer, 用于保存 partial rollout 或过滤后可复用样本。

buffer 的存在很重要。动态采样和长响应训练中, 部分样本可能由于 reward 分布、长度、truncation 或环境失败而不适合直接训练。完全丢弃会浪费昂贵的生成成本; 无条件使用又会污染梯度。slime 通过 filter 与 buffer 把这部分决策显式化。

rollout 后处理

RolloutManager 会把 Sample 转换为训练数据。转换字段包括 tokens、response_lengths、rewards、raw_reward、truncated、sample_indices、loss_masks、rollout_log_probs、routed experts、metadata、multimodal inputs 和 teacher_log_probs。若启用 `--balance-data`, 框架会使用 sequence length balanced partition, 尽量让数据并行 rank 的 token 负载接近; 否则按数据并行 rank 做 strided split。

奖励后处理同样在 rollout manager 中完成。对于 GRPO、GSPO 和 Reinforce++ baseline, 若启用 reward normalization, 会按 group 做 reward 标准化; GRPO/GSPO 还可按标准差归一化。这一步直接影响 advantage 的尺度, 进而影响 policy gradient 的稳定性。

| 训练端计算

训练端的核心问题，是把 rollout 样本变成可微的策略更新。slime 在 Megatron 内部计算当前策略 logprob、reference logprob、teacher logprob、优势、returns、KL、policy loss、value loss 或自定义 loss，并在分布式并行下保持张量形状与统计口径一致。

logprob 与 entropy

slime/backends/megatron_utils/loss.py 中的 get_log_probs_and_entropy 会基于 packed token layout 计算 shifted token 的 logprob 与 entropy。大模型训练通常在 tensor parallel 下使用 vocab parallel cross entropy；若启用 context parallel，还要处理 sequence 维度的切分与 all-gather。源码中对 allgather_cp、padding 和 loss mask 的处理，正是为了保证 logprob 与实际参与训练的 response token 对齐。

优势估计

slime/utils/ppo_utils.py 提供优势与 returns 计算。不同 estimator 的语义不同：

Estimator	训练信号	适用理解
grpo	按 prompt group 的 reward 相对差异形成 token 级优势。	适合每个 prompt 采样多个 response 的 verifier 任务，省去显式 value model。
gspo	与 GRPO 类似，但按 GSPO 口径处理策略目标。	适合需要 group-relative policy optimization 的变体实验。
ppo	基于 reward、KL 和 value function 计算 GAE 与 returns。	需要 critic，训练链路更完整，资源也更重。
reinforce_plus_plus	使用折扣 returns。	较直接的 policy gradient 路径。
reinforce_plus_plus_baseline	在 group baseline 基础上减小方差。	适合不启用 critic 但需要 baseline 的任务。

KL 通过 compute_approx_kl 计算。若启用 OPD，框架还会使用 teacher reverse-KL 修正 advantage；若启用跨数据并行 advantage normalization，统计需要在 DP 维度保持一致。这里的细节决定了实验可复现性和指标解释，不应只看最终 reward 曲线。

policy loss

policy_loss_function 会读取当前 logprob、旧策略或 rollout logprob、优势和 loss mask，计算 PPO/GSPO 风格 clipping、KL loss、entropy、ratio、clip ratio 和可能的 mismatch metrics。slime 还提供 OPSM mask、TIS/RS correction、自定义 reducer 和自定义 loss hook。

对工程使用者而言，最容易出错的是三件事。第一，loss mask 必须精确覆盖 response 中应该学习的 token。第二，rollout logprob 与当前策略 logprob 的版本关系必须清楚，特别是异步 rollout 或权重更新间隔大于 1 时。第三，reward scale 与 advantage normalization 会改变有效学习率，不能在不同实验之间随意比较。

critic 与 value loss

当 estimator 需要 value function 时, slime 会创建 critic 并计算 value loss。critic 的资源开销不可忽略, 因为它同样运行在 Megatron 训练后端中, 并可能与 actor 共享部分配置。若任务采用 GRPO 或其他不需要 critic 的方法, 可以减少一部分训练复杂度, 但也失去了 value baseline 的表达能力。

SFT 与自定义 loss

`loss_function` 可以分派到 policy loss、value loss、SFT loss 或 custom loss。自定义 loss 不是为了把所有算法逻辑写进外部脚本, 而是提供一个受控扩展点: 保留 slime 的数据、并行、权重同步和 rollout 框架, 只替换任务所需的局部目标函数。

权重同步与显存切换

slime 与普通离线训练框架最大的差别之一，是训练权重必须频繁同步给推理引擎。SGLang rollout server 不能长期停留在初始 checkpoint，否则生成样本来自旧策略，RL 训练语义会失真。slime 为 colocate 与 decoupled 两种资源布局分别实现了权重更新路径。

decoupled: UpdateWeightFromDistributed

在训练与 rollout 使用不同 GPU 的 decoupled 模式下，UpdateWeightFromDistributed 通过 NCCL group 将 Megatron 权重广播给 SGLang engines。源码会按 pipeline parallel rank 创建类似 `slime-pp_{pp_rank}` 的通信组，由训练 rank 与 rollout engine GPU 共同参与。非 expert 权重和 expert 权重分开处理；tensor parallel 需要 all-gather；expert parallel 需要按 expert 维度聚合；最后转换为 Hugging Face 风格张量名与切片。

为了避免 broadcast 死锁，rollout engines 在更新期间需要 lock。大张量会按 `update_weight_buffer_size` 分桶，metadata 通过 Ray 传递，tensor 数据通过 NCCL broadcast。这个路径适合训练与 rollout 并行占用独立资源的场景，代价是通信拓扑和版本控制更复杂。

colocate: UpdateWeightFromTensor

colocate 模式下，训练与 rollout 共享同一批 GPU，不能简单让 SGLang 一直持有全部显存。

UpdateWeightFromTensor 会把 HF 格式权重 chunk 从 GPU 转到 CPU，序列化为 flattened buckets，再使用 `Gloo gather_object` 在 colocated ranks 间收集，随后通过 Ray IPC 调用 SGLang engine 的 `update_weights_from_tensor`。若 rollout engine 中仍有远端部分，也会处理分布式更新。

colocate 的优点是节省 GPU 资源，适合中小规模或资源紧张实验。缺点是训练和 rollout 不能完全并行，显存 release/resume/offload 的正确性成为关键。官方 GLM4.5 发布说明中提到，Megatron offload 可将 residual GPU memory 从 15-18GB 降到 3-5GB，使 rollout 侧可以使用更高的 SGLang memory fraction，这正是 colocate 路径的典型价值。

参数重组: common.py

`update_weight/common.py` 中的 `all_gather_param` 处理 Megatron 参数到 rollout 权重的重组。例如 tensor parallel shard 需要 all-gather，GLU 的 `linear_fc1` 需要重新切分，部分 grouped MoE 的 `linear_fc2` 维度还要修正。权重同步不能被理解为“把 state_dict 发给 SGLang”，因为训练态权重布局与推理态权重布局并不相同。

显存切换与低精度 rollout

低精度是 slime 扩展大模型 RL 的重要手段。官方文档介绍了 FP8 rollout、FP8 training、INT4 QAT 与 INT4 inference。FP8 rollout 通常需要通过 `tools/convert_hf_to_fp8.py` 生成 blockwise quantized HF checkpoint；FP8 training 使用 TransformerEngine。需要注意，FP8 training 下保存的 `torch_dist` checkpoint 通常仍以原始精度保存，若要用 FP8 推理评估，需要先转回 HF，再转换为 FP8 HF。

在 GLM4.5 发布说明中，官方报告了 fp8 rollout、DeepEP、MTP、offload、CPU Adam 和更快权重同步等优化方向。对于 GLM4.5 355B-A32B，fp8、DeepEP 与 speculative sampling 将单数据点速度从低于 10 token/s 提升到 60-70 token/s；bf16 权重同步可达 48 秒，fp8 blockwise quantization 与 update 在当时分支约 100 秒。此类数字不应机械外推到其他模型，但它说明 slime 的优化目标是大模型 RL 的系统瓶颈，而不是单个 kernel benchmark。

扩展与生产要点

slime 的正确用法，是尽量保留框架主干，把任务差异放入明确扩展点。这样既能复用 Megatron、SGLang、Ray、权重同步和数据切分，又能把 reward、生成、过滤、loss 和评估逻辑按任务替换。

扩展接口速查

接口参数	插入位置	典型用途
--rollout-function-path	整轮 rollout	完全替换 rollout 生成流程。
--custom-generate-function-path	单样本生成	多轮对话、工具调用、环境交互、自定义采样。
--custom-rm-path	reward model	verifier、规则奖励、执行奖励、外部服务奖励。
--dynamic-sampling-filter-path	rollout 中	按 group reward 或响应状态筛选样本。
--buffer-filter-path	data buffer	决定 partial rollout 或失败样本是否回收。
--rollout-sample-filter-path	rollout 后	过滤单个 Sample。
--rollout-all-samples-process-path	rollout 后	对全部样本做统一处理或统计。
--custom-loss-function-path	Megatron loss	替换或扩展训练目标。
--custom-tis-function-path	policy loss 修正	自定义 importance sampling 或 ratio 修正。
--custom-convert-samples-to-train-data-path	数据转换	特殊 multimodal、tool trace 或额外训练字段。
--data-source-path	数据源	自定义 prompt 读取、在线数据或课程学习。
--eval-function-path	评估	任务特定 evaluation。

SGLang 配置文件

当 rollout 需要多个模型、reference server、reward model server、encoder 或 PD disaggregation 时，建议使用 SGLang YAML 配置。顶层 `sglang` 下可以定义多个 model，每个 model 拥有自己的 router、model path、是否 update weights、单 engine GPU 数和 server groups。源码中可见 group 类型包括 regular、prefill、decode、placeholder 和 encoder。自定义代码可通过 `get_model_url(args, "ref")` 获取某个模型的 URL。

PD disaggregation 将 prefill 与 decode server 分离，适合长上下文、多轮、agentic RL 和 prefill 成本突出的场景。它不是默认应该打开的开关；只有当 profile 显示 prefill/decode 资源形态明显不同，且 router 与 server group 配置能稳定运行时，才值得引入。

容错与健康检查

`--use-fault-tolerance` 会启用 rollout 健康检查。slime 通过 `/health_generate` 监控 SGLang server；如果发现 server 死亡，会在当前 rollout 与权重更新之后停止并重启相关服务。关键参数包括 `rollout-health-check-first-wait`、`rollout-health-check-interval` 和 `rollout-health-check-timeout`。

这类容错适合长时间训练，但不能替代根因排查。若 server 经常死亡，应优先查看显存峰值、请求长度、router worker 状态、NCCL 日志、weight update bucket、CUDA IPC cache、模型 processor 和自定义生成函数。v0.2.4 release 中修复了权重更新期间 CUDA IPC cache leak，以及 FLA 层在 SP/CP 下的梯度膨胀问题，这说明 rollout 与训练的交界处是高风险区域。

调试路径

问题	优先检查	解释
启动失败	Ray resource、端口、GPU 数、checkpoint 路径、SGLang 参数。	多数启动失败来自资源或模型加载，不是 RL 算法。
首轮 rollout 异常	tokenizer、chat template、prompt key、label key、response length、finish reason。	生成链路先于训练链路暴露数据问题。
reward 全零或 NaN	custom rm、verifier、metadata、截断、loss mask。	reward 不可信时，policy loss 的数值没有解释价值。
KL 异常	reference checkpoint、rollout logprob、weight version、old actor。	KL 是判断策略漂移和版本关系的核心指标。
OOM	context length、max tokens per GPU、micro batch、SGLang mem fraction、offload。	训练侧和 rollout 侧显存峰值发生在不同阶段，需要分别观测。
吞吐低	最长样本、动态采样、server concurrency、PD disaggregation、weight update 时间。	RL 训练经常被最长 response 和权重同步拖慢，而非平均 token/s。

何时采用 slime

slime 适合三类团队。第一类已经使用或愿意使用 Megatron 训练大模型，并需要把 RL 后训练扩展到多 GPU 或多节点。第二类需要 SGLang server-based rollout，希望在 rollout 中复用结构化输出、多轮交互、router、PD disaggregation 和低精度推理。第三类任务逻辑复杂，需要自定义 reward、生成、过滤、loss、评估和数据源，同时又不想重写训练与权重同步系统。

保持谨慎的场景也很明确。若只是做小模型单卡实验，slime 的分布式复杂度可能超过收益；若团队完全不熟悉 Megatron checkpoint 与并行参数，应先把模型转换和 SFT 训练链路跑稳；若任务 reward 仍不可靠，先投入 verifier 与评估体系，比调 slime 参数更重要。

上手后的工程路线

1. 先按官方 Docker 与示例脚本跑通一个小规模模型，确认生成、reward、训练、保存和权重同步全链路可用。
2. 固定模型与数据，逐项修改 batch、并行、context length 和 SGLang memory fraction，建立本地资源预算表。
3. 引入自定义 reward，不改主训练循环，先用 debug rollout 验证 Sample 字段和 reward 分布。
4. 引入自定义 generate 或 data source，保留 loss 与 weight update 主干，确认失败样本、partial rollout 和 buffer 策略。
5. 在算法层比较 GRPO、GSPO、PPO 或 Reinforce++ 变体，所有实验记录 reward normalization、loss mask、KL 与 weight version。
6. 最后再考虑低精度、PD disaggregation、异步训练和容错。这些是性能与可靠性优化，不应早于正确性验证。

CLOSING POSITION

slime 的工程价值，来自它承认 RL 后训练是一个系统问题。算法目标、生成服务、数据状态、分布式训练、权重版本和资源切换彼此耦合。掌握 slime 的关键，不是记住更多参数，而是能判断每个参数在训练闭环中的位置，以及它改变的是样本语义、训练数学、推理吞吐、显存峰值还是通信成本。

附录：参数速查

下表不是完整 CLI 文档，而是阅读 slime 脚本时最常遇到、也最容易误解的参数索引。完整参数应以官方仓库和当前版本源码为准。

领域	参数	一句话解释
资源	<code>--actor-num-nodes</code>	Megatron actor 使用的节点数量。
资源	<code>--actor-num-gpus-per-node</code>	每节点训练 GPU 数。
资源	<code>--rollout-num-gpus</code>	SGLang rollout 使用的 GPU 总数。
资源	<code>--rollout-num-gpus-per-engine</code>	单个 SGLang engine 的 GPU 数。
资源	<code>--colocate</code>	训练与 rollout 共享 GPU，通过显存切换复用资源。
批量	<code>--rollout-batch-size</code>	每轮取多少个 prompt group。
批量	<code>--n-samples-per-prompt</code>	每个 prompt 采样多少个 response。
批量	<code>--global-batch-size</code>	Megatron 每轮训练消费的全局样本数。
批量	<code>--num-steps-per-rollout</code>	每轮 rollout 后训练多少步。
SGLang	<code>--hf-checkpoint</code>	SGLang 初始化和 tokenizer 来源。
SGLang	<code>--sglang-context-length</code>	rollout 最大上下文长度。
SGLang	<code>--sglang-mem-fraction-static</code>	SGLang static memory fraction。
checkpoint	<code>--ref-load</code>	reference model 的 Megatron checkpoint。
checkpoint	<code>--load</code>	actor 初始化或恢复训练 checkpoint。
checkpoint	<code>--save</code>	训练 checkpoint 保存目录。
算法	<code>--advantage-estimator</code>	选择 GRPO、GSPO、PPO、Reinforce++ 等优势估计。
算法	<code>--use-kl-loss</code>	在训练目标中加入 KL loss。
算法	<code>--kl-loss-coef</code>	KL loss 系数。
算法	<code>--eps-clip</code>	PPO/GRPO 风格 ratio clipping 下界相关参数。
算法	<code>--eps-clip-high</code>	上界 clipping 参数，常用于非对称 clip。
数据	<code>--input-key</code>	JSONL 中 prompt 字段名。
数据	<code>--label-key</code>	JSONL 中 label 字段名。
数据	<code>--apply-chat-template</code>	是否用 tokenizer chat template 包装输入。
可靠性	<code>--use-fault-tolerance</code>	启用 rollout server 健康检查和恢复。
扩展	<code>--custom-rm-path</code>	自定义 reward model 或 verifier。
扩展	<code>--custom-generate-function-path</code>	自定义单样本生成逻辑。

资料来源、复核口径和本地抓取清单见同目录 `sources.md` 与 `data/source-map.tsv`。